

"Given the test function below, implement the class..."

①. void testStack()

```

{
    Stack<std::string> s{2};
    assert(s.getMaxCapacity() == 2);
    try {
        s = s + "examination";
        s = s + "oop";
        s = s + "test";
    }
    catch (std::exception& e) {
        assert(strcmp(e.what(), "Stack is full!") == 0);
    }
    assert(s.pop() == "oop");
    assert(s.pop() == "examination");
}
    
```

Stack is a
 => template class
 => constructor receives max capacity
 => int getMaxCapacity() const;
 => overloaded operator + (not push)
 => Stack operator+(const T& elem)
 => third insertion must fail
 try to add elem to full stack
 => throw exception
 => T pop();

STACK:

```

#include <stdexcept>
#include <string>
#include <cassert>
    
```

generic class [template <typename T>]
 class Stack {

```

Stack<std::string>
Stack<int>
Stack<double>
    
```

private:

```

T* elements;
int size;
int maxCapacity;
    
```

public:

CONSTRUCTOR

```

Stack(int capacity): size{0}, maxCapacity{capacity}
{
    this->elements = new T[this->maxCapacity];
}
    
```

COPY CONSTRUCTOR

```

Stack(const Stack& other): size{other.size}, maxCapacity{other.maxCapacity}
{
    this->elements = new T[this->maxCapacity];
    for(int i=0; i<this->size; i++)
    {
        this->elements[i] = other.elements[i];
    }
}
    
```

ASSIGNMENT OPERATOR

```

Stack& operator=(const Stack& other)
{
    if (this == &other)
        return *this;
    delete[] this->elements;
    this->size = other.size;
    this->maxCapacity = other.maxCapacity;
    this->elements = new T[this->maxCapacity];
    for(int i=0; i<this->size; i++) this->elements[i] = other.elements[i];
    return *this;
}
    
```

①

```

~Stack()
{
    delete[] this->elements;
}

int getMaxCapacity() const
{
    return this->maxCapacity;
}

```

```

Stack operator+ (const T& elem)
{
    if (this->size == this->maxCapacity)
    {
        throw std::runtime_error("Stack is full!");
    }
    this->elements[this->size] = elem;
    this->size++;
    return *this;
}

```

```

T pop()
{
    if (this->size == 0)
        throw std::runtime_error("Empty stack!");
    this->size--;
    return this->elements[this->size];
}
};

```

② void TodoList()

{
 Todo <Activity> todo{}; → template class

Activity tiff { "go to TIFF movie", "20:00" };
 todo += tiff; → overload operator "+"

Activity project { "present project", "09.20" };
 todo += project;

for (auto a: todo) → Todo must be iterable ⇒ range-based loop ⇒ begin() end()

std::cout << a << '\n'; → overload operator "<<"

todo.reversePrint(std::cout); → (std::ostream & out)

}

Activity:
 class Act

{parameter,

→ all objects are statically allocated

class Activity {

private:

std::string description;

std::string time;

public:

Activity(const std::string& description, const std::string& time): ~~desc~~
description {description}, time {time} {}

{

std::string getDescription() const

{ return this->description;

}

std::string getTime() const

{ return this->time;

}

friend std::ostream& operator<<(std::ostream& out, const Activity& a)

{ std::string printedTime = a.time;

for (char& ch : printedTime)

{ if (ch == ':')

{ ch = '.';

}

}

out<< "Activity " << a.description << " will take place at " << printedTime
<< ". ";

return out;

}

};

template <typename T>

class ToDo {

private:

std::vector<T> items;

public:

ToDo() = default;

ToDo<T>& operator+=(const T& elem){

this->items.push_back(elem);

return *this;

}

typename std::vector<T>::iterator begin(){

return this->items.begin();

}

typename std::vector<T>::iterator end(){

return this->items.end();

}

] → for (auto& a : todo)
{ a can be modified
}

```

typename std::vector<T>::const_iterator begin() const {
    return this->items.begin();
}
typename ... end() const {
    ...
}

```

```

void reversePrint(std::ostream & out) const {
    for (auto it = this->items.rbegin(); it != this->items.rend(); ++it)
    {
        out << *it << '\n';
    }
};

```

③ / 2018 - Row 2

```

void testFct()
{
    vector<int> v1 {4, 2, 1, -4};
    assert(fct<int>(v1) == 3);
    vector<int> v2;
    try {
        fct<int>(v2);
        assert(false);
    }
    catch (std::exception &)
    {
        assert(true);
    }
    vector<double> v3 {2, 10.5, 5, -10};
    assert(fct<double>(v3) == 7.5);
    vector<string> v4 {"y", "g", "a", "u"};
    assert(fct<string>(v4) == "ygam");
}

```

```

template <typename T>
T fct(const vector<T> & v)
{
    if (v.empty()) {
        throw runtime_error("Empty vector!");
    }
    T result = v[0];
    for (size_t i = 1; i < v.size(); i++)
    {
        result = result + v[i];
    }
    return result;
}

```

STL vector

int: 0
double: 0.0
string: ""

④ Row 2

```

void function2() {
    Adder<int> add(5);
    add = add + 5 + 2;
    ++add; // adds last added value again
    add + 8;
    cout << add.sum() << "\n";
    --add;
    cout << add.sum() << "\n";
    --(--add);
    cout << add.sum() << "\n";
    try {
        --(--(--add));
    }
    catch (runtime_error ex) { cout << ex.what(); }
}

```

```

template <typename T>
class Adder {
private:
    vector<T> values;
public:
    Adder(const T& initialValue) {
        values.push_back(initialValue);
    }
    Adder<T> & operator+ (const T& value) {
        values.push_back(value);
        return *this;
    }
}

```

②

```

Adder<T> & operator++() {
    values.push_back(values.back());
    return *this;
}

```

M A T V

TAMV

```

Adder<T> & operator--() {
    if(values.size() <= 1) {
        throw runtime_error("No more values!");
    }
    values.pop_back();
    return *this;
}

```

Hi An Tu Va

Hi Tu VAn

```

T sum() const {
    T result = values[0];
    for(size_t i=1; i<values.size(); i++) {
        result = result + values[i];
    }
    return result;
}

```

};

⑤ void complex() {

Complex a{ }, b{1,2}, c{6,4}, d{b};

assert(a.getReal() == 0 && a.getImaginary() == 0);

assert(c.getImaginary() == 4);

assert(b == d);

Complex res1 = c/2;

cout << res1 << "\n";

try {

Complex res2 = b/0;

} catch (runtime_error & e) {

assert(strcmp(e.what(), "Div by zero!") == 0);

}

vector<string> v1 { std::vector<string> {"hello", "bye"} };

v1.printAll(std::cout);

vector<Complex> v2 { std::vector<Complex> {a,b,c,d} };

v2.printAll(std::cout);

}

default constructor
constructor w params

copy constructor

friend ostream& operator<<(ostream& out, const Complex& c)

```
class Complex {
```

```
private:
```

```
int real;
```

```
int imaginary;
```

```
public:
```

```
Complex(): real{0}, imaginary{0} {}
```

```
Complex(int real, int imaginary): real{real}, imaginary{imaginary}
```

```
Complex(const Complex& other): real{other.real}, imaginary{other.imaginary}
```

```
int getReal() const {
```

```
    return this->real;
```

```
}
```

```
int getImaginary() const {
```

```
    return this->imaginary;
```

```
}
```

```
bool operator==(const Complex& other) const {
```

```
    return this->real == other.real &&
```

```
        this->imaginary == other.imaginary;
```

```
}
```

```
Complex operator/(int value) const {
```

```
    if (value == 0) {
```

```
        throw runtime_error("Div by 0!");
```

```
    }
```

```
    return Complex(this->real / value, this->imaginary / value);
```

```
}
```

```
friend ostream& operator<<(ostream& out, const Complex& c) {
```

```
    out << c.real << "+" << c.imaginary << "i";
```

```
    return out;
```

```
}
```

```
};
```

```
template <typename T>
```

```
class Vector {
```

```
private:
```

```
std::vector<T> elements;
```

```
public:
```

```
Vector(const std::vector<T>& elems): elements{elems} {}
```

```
void printAll(const std::ostream& out) const {
```

```
    for (const auto& elem : this->elements) {
```

```
        out << elem << " ";
```

```
    }
```

```
}
```

```
};
```